AWS IAM Policy Configuration Profiles dataset for Cloud Misconfiguration Detection

Dataset : AWS IAM Policy Configuration Profiles Dataset for Cloud Misconfiguration Detection

Author / Dataset Creator: MD Owes Quruny Shubho
ORCID: https://orcid.org/0009-0002-4870-5280
Contact: owes.shubho@gmail.com
Author contributions:
Version: 1.0 | **Date:** July 11, 2026
Format: JSON (nested) and CSV (flattened feature matrix)
Size: 1,000 labeled configuration profiles
Class balance: 700 compliant (70%) / 300 misconfigured (30%) **Reproducibility:** deterministic; fixed random seed = 42

Table of Contents

PART A — Dataset Information 3
A.1 Specifications Table 3
A.2 File Inventory 3
A.3 Data Dictionary 4
A.4 Vulnerability Catalog 6
A.5 Summary Statistics 6
PART B — Data Generation Process 8
B.1 Design Principles 8
B.2 Overall Pipeline 8
B.3 Step 1 — Contextual Attribute Sampling 8
B.4 Step 2 — Compliant Base Policy Construction 9
B.5 Step 3 — Vulnerability Injection (Misconfigured Profiles Only) 9
B.6 Step 4 — Feature Extraction 10
B.7 Step 5 — Serialization and Summary 10
B.8 Reproduction 10
B.9 Notes, Assumptions, and Limitations 11
Contact 11

PART A — Dataset Information

A.1 Specifications Table

Item	Description
Author	MD Owes Quruny Shubho
ORCID	0009-0002-4870-5280
Contact email	owes.shubho@gmail.com
Subject area	Computer Science — Cloud Security / Machine Learning
Specific subject	Identity and Access Management (IAM) policy misconfiguration detection
Data type	Synthetic (programmatically generated), labeled
Format	JSON (raw policies + metadata), CSV (extracted features), JSON (summary)
Number of records	1,000 configuration profiles
Class labels	Binary: `0` = compliant, `1` = misconfigured
Auxiliary labels	Multi-label vulnerability tags (11 types) on misconfigured records
Class distribution	700 compliant / 300 misconfigured (70/30)
Feature count (CSV)	21 model-ready features + identifiers and labels (25 columns total)
Policy language	AWS IAM policy grammar, version `2012-10-17`
Generation method	Template-based synthesis with controlled vulnerability injection
Randomization	Seed-controlled (seed = 42); fully reproducible
Accompanying code	`generate_iam_dataset.py` (Python 3, standard library only)

A.2 File Inventory

File	Format	Description
`iam_dataset.json`	JSON	Full dataset. Each record holds the raw IAM policy document, context metadata, and labels. Source of truth.
`iam_dataset.csv`	CSV	Flattened feature matrix. One row per profile with extracted numeric/binary features and the label. Model-ready.
`dataset_summary.json`	JSON	Descriptive statistics: class balance, vulnerability distribution, feature-column list.
`generate_iam_dataset.py`	Python	The generator script that produces all three files above.

A.3 Data Dictionary

A.3.1 Profile-level fields (`iam_dataset.json`)

Field	Type	Description
`profile_id`	string (UUID)	Unique identifier for the configuration profile. Join key to the CSV.
`entity_name`	string	Synthetic IAM entity name encoding department, environment, and type.
`entity_type`	categorical	Entity the policy is attached to: `role` or `user`.
`policy_name`	string	Human-readable policy name.
`created_at`	string (ISO 8601)	Synthetic creation timestamp.
`context`	object	Surrounding IAM/account context (see A.3.2).
`policy_document`	object	The AWS IAM policy document in native JSON (see A.3.3).
`label`	integer	Ground-truth class: `0` = compliant, `1` = misconfigured.
`label_text`	string	Human-readable label: `compliant` / `misconfigured`.
`vulnerabilities`	list of strings	Vulnerability tags present; empty for compliant profiles.

A.3.2 Context object fields (`context`)

Field	Type	Description
`AccountId`	string	Synthetic 12-digit AWS account identifier.
`Environment`	categorical	Deployment environment: `prod`, `staging`, `dev`, `sandbox`.
`MFAEnabled`	boolean	Whether MFA is enabled on the entity's account.
`AccessKeyAgeDays`	integer	Access key age in days (key-rotation hygiene proxy).
`LastUsedDays`	integer	Days since credential last used (dormant-credential proxy).
`PolicyType`	categorical	Policy attachment type: `managed` or `inline`.

A.3.3 Policy document elements (`policy_document`)

Standard AWS IAM policy structure. Each statement may contain:

Element	Type	Description
`Version`	string	Policy language version, always `2012-10-17`.
`Statement`	list	One or more permission statements.
`Sid`	string	Statement identifier.
`Effect`	string	`Allow` or `Deny`.
`Action` / `NotAction`	string or list	Service actions granted (or excluded).

`Resource`	string or list	Target ARN(s), or * for all resources.
`Principal`	string or object	Entity granted access (present in resource/trust policies).
`Condition`	object	Contextual constraints (MFA, source IP, encryption, etc.).

A.3.4 Extracted feature fields (`iam_dataset.csv`)

Binary features take values in `{0, 1}`. Identifier and analysis-only columns are marked; these are normally dropped before model training.

#	Feature	Type	Description
1	`profile_id`	string	*Identifier.* Join key to the JSON record.
2	`entity_name`	string	*Identifier.* Synthetic entity name.
3	`entity_type`	categorical	`role` or `user`.
4	`environment`	categorical	Deployment environment.
5	`num_statements`	integer	Number of statements in the policy.
6	`num_allow`	integer	Count of `Allow` statements.
7	`num_deny`	integer	Count of `Deny` statements.
8	`has_wildcard_action`	binary	A statement uses * or a `service:*` wildcard action.
9	`has_wildcard_resource`	binary	A statement applies to `Resource: "*"`.
10	`has_full_admin`	binary	An `Allow` grants * action on * resource.
11	`has_public_principal`	binary	A statement sets `Principal: "*"`.
12	`has_passrole_wildcard`	binary	`iam:PassRole` granted on `Resource: "*"`.
13	`priv_esc_action_count`	integer	Number of privilege-escalation-capable actions granted.
14	`has_mfa_condition`	binary	A statement enforces `aws:MultiFactorAuthPresent`.
15	`has_ip_condition`	binary	A statement restricts by `aws:SourceIp`.
16	`has_encryption_condition`	binary	A statement enforces server-side encryption.
17	`num_conditions`	integer	Number of statements carrying a `Condition` block.
18	`uses_notaction`	binary	A statement uses the `NotAction` element.
19	`total_action_count`	integer	Total action strings across all statements.
20	`cross_account_trust`	binary	A statement grants access to an external AWS principal.
21	`mfa_enabled`	binary	Account-level MFA flag (from context).
22	`access_key_age_days`	integer	Access key age (from context).

23	`is_prod`	binary	Whether the environment is `prod`.
24	`vulnerabilities`	string	*Analysis only.* Pipe-delimited vulnerability tags.
25	`label`	integer	Ground-truth class label.

A.4 Vulnerability Catalog

The 11 vulnerability types that may be present in misconfigured profiles.

Vulnerability tag	Meaning	Security risk
`full_admin`	`Allow` with `Action: "*"` on `Resource: "*"`	Effective administrator access; violates least privilege.
`wildcard_action`	Service-wide wildcard action (e.g. `s3:*`)	Over-broad permissions beyond operational need.
`wildcard_resource`	Statement applies to `Resource: "*"`	Permissions extend to all resources.
`public_principal`	`Principal: "*"` with no restricting condition	Public / anonymous access.
`passrole_wildcard`	`iam:PassRole` on `Resource: "*"`	Privilege-escalation vector.
`privilege_escalation`	Sensitive `iam:*` / `sts:*` write actions granted	Self-elevation of privileges.
`no_mfa_on_privileged`	Destructive actions without MFA condition	Sensitive operations without strong auth.
`missing_encryption`	`s3:PutObject` without enforced encryption	Data may be stored unencrypted.
`no_condition_constraints`	All `Condition` blocks stripped	Removes contextual guardrails.
`cross_account_no_externalid`	Cross-account trust without `ExternalId`	Confused-deputy exposure.
`notaction_allow`	`Allow` with `NotAction` over `Resource: "*"`	Implicit over-permission.

A.5 Summary Statistics

Class balance

Class	Label	Count	Proportion
Compliant	0	700	70.0%
Misconfigured	1	300	30.0%

Vulnerability distribution (across 300 misconfigured profiles; tags are non-exclusive because some flaws imply correlated secondary tags)

Tag	Occurrences
`wildcard_resource`	200
`wildcard_action`	83
`no_condition_constraints`	50
`public_principal`	49
`missing_encryption`	48
`privilege_escalation`	47
`notaction_allow`	46
`full_admin`	43
`no_mfa_on_privileged`	41
`cross_account_no_externalid`	39
`passrole_wildcard`	39

Vulnerability tags per misconfigured profile

Tags	Profiles
1	89
2	77
3	97
4	34
5	3

Statements per policy (all 1,000 profiles)

Statements	Profiles
1	133
2	285
3	322
4	186
5	56
6	16
7	2

Contextual attributes are approximately uniform across the four environments and across entity types (~53% roles, ~47% users).

PART B — Data Generation Process

This part describes, in detail, how each profile was produced. The full logic is implemented in `generate_iam_dataset.py` using only the Python standard library, so the dataset can be regenerated on any machine with no external dependencies.

B.1 Design Principles

Three principles governed the generation procedure:

1. **Structural realism.** Every policy conforms to the AWS IAM policy grammar (`Effect`, `Action`, `Resource`, `Principal`, `Condition`; version `2012-10-17`), so features learned on this data correspond to real policy constructs.
2. **Ground-truth labeling by construction.** A profile's label is fixed by the path used to build it, not by a post-hoc scan. This eliminates labeling noise and yields exact vulnerability annotations.
3. **Controlled, non-trivial separability.** Compliant and misconfigured policies are drawn from the same template distribution (same services, resource naming, environments). Vulnerabilities are injected into otherwise plausible policies rather than generated as visibly distinct artifacts.

B.2 Overall Pipeline

```
specify n, seed, compliant_ratio
        │
        ▼
build label vector  (round(n×ratio) zeros, rest ones)  →  shuffle
        │
        ▼
for each label:
        ├─ draw context (department, environment, entity type, account)
        ├─ build a compliant base policy  (least-privilege templates)
        └─ if label == misconfigured:  inject 1–3 vulnerabilities
        │
        ▼
shuffle all profiles
        │
        ▼
serialize →  iam_dataset.json
extract features →  iam_dataset.csv
compute stats →  dataset_summary.json
```

All stochastic choices derive from a single seeded generator (`random.seed(seed)`), so identical parameters reproduce the dataset exactly.

B.3 Step 1 — Contextual Attribute Sampling

For each profile the following are drawn:

- **Department** — one of `data, web, billing, ops, ml, security, analytics`.

- **Environment** — one of `prod, staging, dev, sandbox`.
- **Entity type** — `role` or `user`.
- **Account context** — `AccountId` (random 12 digits), `MFAEnabled` (true with probability ≈ 0.85), `AccessKeyAgeDays` (1–400), `LastUsedDays` (0–180), `PolicyType` (`managed/inline`).

Department and environment feed into resource ARNs so that generated ARNs are internally consistent (e.g. `arn:aws:s3:::billing-prod-bucket/*`).

B.4 Step 2 — Compliant Base Policy Construction

Every profile — compliant or not — starts from a least-privilege base policy, built as follows:

1. Select 1–3 AWS services at random from `s3, ec2, dynamodb, lambda, rds, logs`.
2. For each service, create an `Allow` statement that grants **scoped read actions**, optionally augmented with a small set of **write actions**.
3. Target a **specific resource ARN** (never a wildcard) built from the department and environment.
4. Attach security-enhancing `Condition` blocks where appropriate:
   - **S3 writes** → enforce `s3:x-amz-server-side-encryption = AES256`.
   - **Production** → optionally restrict `aws:SourceIp` to a corporate range.
   - **Production writes/deletes** → require `aws:MultiFactorAuthPresent = true`.
5. With probability ≈ 0.4, append an explicit `Deny` statement blocking non-TLS access (`aws:SecureTransport = false`) as a defense-in-depth measure.

For a compliant profile (label 0) this policy is the final artifact and the vulnerability list is empty.

B.5 Step 3 — Vulnerability Injection (Misconfigured Profiles Only)

For a misconfigured profile (label 1), the base policy is mutated:

1. Draw the **number of vulnerabilities** to inject: 1, 2, or 3 with probabilities 0.50, 0.35, 0.15. Most misconfigured profiles therefore carry a single dominant flaw; a minority are compound.
2. Sample that many **distinct** vulnerability types (without replacement) from the catalog in A.4.
3. Apply each mutation to the policy. Representative mutations:
   - `full_admin` → append `Allow *:*`.
   - `wildcard_action` → append `Allow service:*` on a scoped resource.
   - `wildcard_resource` → set an existing statement's resource to `*`.
   - `public_principal` → append a statement with `Principal: "*"`.
   - `passrole_wildcard` → append `Allow iam:PassRole` on `*`.
   - `privilege_escalation` → append 2–4 sensitive `iam:*/sts:*` actions.

- `no_mfa_on_privileged` → append destructive actions with no MFA condition.
- `missing_encryption` → append `s3:PutObject` with no encryption condition.
- `no_condition_constraints` → strip all `Condition` blocks.
- `cross_account_no_externalid` → append external trust with no `ExternalId`.
- `notaction_allow` → append `Allow` with `NotAction` over *.

Correlated secondary tags

Some mutations logically imply others and record multiple tags. For example, `full_admin` implies both `wildcard_action` and `wildcard_resource`, so all three are recorded. Consequently the number of vulnerability *tags* on a profile can exceed the number of injection *operations* — which is why some profiles show 4–5 tags in A.5 even though at most 3 operations were applied. This mirrors how a real scanner reports several overlapping findings for one insecure statement.

B.6 Step 4 — Feature Extraction

To produce the CSV matrix, each raw policy document is reduced to the 21 features in A.3.4 by a deterministic extraction function that:

- counts statements and separates `Allow` / `Deny` effects;
- flags wildcard actions (* or `service:*`), wildcard resources, and the full-admin combination;
- detects `Principal: "*"`, external-principal trust, and `iam:PassRole` on *;
- counts privilege-escalation actions against a fixed sensitive-action list;
- inspects `Condition` blocks for MFA, source-IP, and encryption enforcement;
- tallies conditions, total actions, and `NotAction` usage;
- appends context-derived features (`mfa_enabled`, `access_key_age_days`, `is_prod`).

Feature extraction reads only the policy and context, never the label, so no label information leaks into the features beyond what the policy structure itself implies.

B.7 Step 5 — Serialization and Summary

The generator writes:

- **`iam_dataset.json`** — all profiles with raw policies and labels;
- **`iam_dataset.csv`** — the extracted feature matrix;
- **`dataset_summary.json`** — class balance, vulnerability distribution, and the feature-column list.

B.8 Reproduction

```
python3 generate_iam_dataset.py --n 1000 --seed 42 --compliant-ratio 0.7
```

Parameters:

Flag	Meaning	Default
`--n`	Total number of profiles	1000
`--seed`	Random seed controlling all stochastic choices	42
`--compliant-ratio`	Fraction of profiles labeled compliant	0.5
`--outdir`	Output directory	current directory

Fixing `--seed` guarantees byte-identical output, enabling exact replication.

B.9 Notes, Assumptions, and Limitations

- **Synthetic origin.** Policies are drawn from a finite template set and do not span the full diversity of production IAM. Models validated only here should be re-validated on real policies before operational use.
- **Clean label–feature mapping.** Because labels are construction-determined, each vulnerability maps cleanly to one or more features; simple classifiers may score very high on the CSV. For a harder task, train directly on the raw `policy_document` in the JSON (e.g. term-frequency or embedding representations).
- **Independent context.** Account-context fields are drawn independently of policy content and should be treated as auxiliary rather than authoritative signals.
- **Evaluation under imbalance.** With a 70/30 split, plain accuracy is misleading (a majority-class predictor scores 70%). Use stratified splitting and report precision, recall, and F1 on the misconfigured class, optionally with class re-weighting.

Contact

Questions regarding the dataset, the generation procedure, or reuse should be directed to the author:

MD Owes Quruny Shubho | ORCID: https://orcid.org/0009-0002-4870-5280
Email: owes.shubho@gmail.com

This dataset is fully synthetic, created for educational, benchmarking, and analytical purposes, and is not affiliated with or endorsed by any company.